Analizador Semántico AST

Rocío Corral[1]

[1]Lic. Cs. de la Computación/Facultad de Ingeniería/UNCuyo

30 de mayo de 2023

Abstract

Decisiones de diseño, metodología, lenguaje utilizado, arquitectura, funcionamiento y resultados del proceso de implementación de un compilador para el lenguaje de programación TinyRust+ se explican en el presente trabajo.

1. Introducción

El objetivo de este informe es explicar y fundamentar la implementación de la etapa de análisis semántico de sentencias del compilador de TinyRust+.

En la primera etapa se implementó el analizador léxico, programa que genera un conjunto de *tokens* a partir de un archivo de texto. Estos tokens se definen utilizando como base las reglas descritas en el manual de TinyRust+ para delimitar palabras reservadas, operadores, identificadores, tipos y demás.

En la segunda etapa se implementó el analizador sintáctico, programa que consume *tokens* generados por el analizador léxico y verifica que el programa cumpla con las reglas de la gramática de TinyRust+. Esta implementación consiste en un analizador sintáctico descendente predictivo recursivo. Para su implementación, se tuvo que tratar la gramática de TinyRust+ descrita en el manual transformándola a notación BNF, eliminando la recursividad a izquierda y factorizando.

En la tercera etapa se implementó la tabla de símbolos y el análisis semántico de declaraciones. La tabla de símbolos se construye de forma intercalada a medida que se realiza el análisis sintáctico del archivo de entrada. Durante el análisis sintáctico se agregan entidades a la tabla y para cuando se ha completado el análisis exitosamente, la tabla de símbolos está completa y se realiza su consolidación.

En la cuarta etapa se implementó el árbol sintáctico abstracto (AST, por sus siglas en inglés) y el análisis semántico de sentencias. Este árbol se construye de forma recursiva en simultáneo con el análisis sintáctico y la construcción de la tabla de símbolos. En el presente informe se explicará exclusivamente la construcción del AST y los detalles de las validaciones que luego se realizan sobre este.

En el presente informe se detallarán decisiones de diseño, metodología, arquitectura, funcionamiento y resultados de la implementación del analizador semántico de sentencias. En algunas secciones también se detallan las decisiones y funcionamiento del analizador semántico de declaraciones, dado que ambos análisis son complementarios. Hacia el final del trabajo se

muestran algunos ejemplos y casos de prueba y se explica cómo ejecutar el proyecto.

2. Metodología

Para el desarrollo de este proyecto, el primer paso fue hacer un análisis de la información disponible en el manual sobre el funcionamiento semántico esperado de TinyRust+ y definir una serie de aspectos de diseño detallados en la siguiente sección. Posterior a esto, se diagramó la arquitectura del AST mediante anotaciones en la gramática de atributos y se listaron las validaciones para cada caso de uso. Finalmente se implementó la construcción recursiva del AST y luego se implementaron las validaciones sobre la estructura y contenido del AST.

El AST se inicializa como una estructura vacía antes de comenzar el análisis sintáctico y durante el análisis sintáctico se construye desde sus hojas hasta la raíz. Luego de que una regla sintáctica haga match, se utiliza la información de los tokens consumidos en esa regla para generar un nodo del AST y se retorna en la regla. Cuando esta regla es *matcheada* en una regla superior, se utiliza el nodo obtenido para construir a su vez el nodo que retornará la regla superior, formando así un árbol de nodos.

De este modo, los primeros nodos que se generan son los de literales y variables, que luego se anexan a los nodos de las expresiones. Los nodos de expresiones son utilizados al armar los nodos de las sentencias; los nodos de sentencias se anexan a los nodos bloque y luego los nodos de bloques se anexan al nodo del método correspondiente. Finalmente, los nodos de método se guardan dentro de los nodos de clase y el nodo raíz del AST contiene únicamente una lista de clases.

Luego de construir el AST por completo y consolidar la tabla de símbolos, se procede a realizar la validación del AST, que se realiza también de forma recursiva, pero esta vez se realiza desde la raíz hasta las hojas del árbol.

La validación del nodo raíz del AST consiste en validar cada nodo clase, mientras que la validación de un nodo clase consiste en validar todos sus métodos. Luego la validación de cada método es más diversa, se utiliza la tabla de símbolos para corroborar que las entidades utilizadas en cada sentencia del método existen, que existan las propiedades accedidas en cada entidad y que se utilicen entidades del tipo correcto en cada contexto (como asignaciones, operaciones, retornos, etc).

2.1 Consideraciones

A continuación se listan una serie de consideraciones y decisiones de diseño semántico relevantes antes de profundizar en la implementación.

- No se permite declarar con el mismo identificador dos atributos, dos métodos, dos argumentos de un método o dos variables dentro de un bloque de método.
- No se permite declarar una variable en el bloque de un método con el mismo nombre que un argumento del método, ya que impediría el acceso al argumento.

- No se permite declarar variables, argumentos, atributos ni métodos utilizando el identificador “self”.
- Sí se permite declarar variables, argumentos y atributos utilizando el identificador “create”. Pero no se permite declarar métodos utilizando dicho identificador.
- Se agregó en la etapa léxica la validación en tiempo de compilación que los literales de tipo I32 no excedan los 32 bits.
- En la tabla de símbolos se guarda registro de las variables de cada bloque de método para su posterior uso por el AST durante sus validaciones.
- Se valida para cada método que el bloque retorna el tipo de dato esperado para el método. Si dentro del bloque hay sub-bloques o ramas, se valida que en cada rama se retorne el tipo esperado.
- Se permite usar instancias de una clase dada en contextos donde se espera una instancia de su superclase. Por ejemplo, al asignar variables o retornar dentro de un método.
- No se permite acceder atributos ni métodos no estáticos de la clase dentro de métodos estáticos.
- No se permite el acceso a atributos privados de una clase en otras clases ni en subclases. Pero **sí** se permite el acceso a atributos privados de otras instancias de la misma clase, comportamiento similar a Java. Por ejemplo, si el constructor de la clase Nodo recibe una instancia distinta de la clase Nodo como argumento, los atributos privados de esa instancia sí son accesibles:

```
class Nodo {
   I32: attrPrivado;
   public Nodo (Nodo: ejemploVar) {
         self.attrPrivado = ejemploVar.attrPrivado; // EXITO.
   }
}
```

- Sí se permite invocar de manera dinámica a métodos estáticos. Este es un comportamiento similar al de Java. Si bien Java da una advertencia al respecto, no es un error. Un ejemplo válido sería:

```
class Ejemplo {
   static fn metodoEstatico() -> void {}
   fn metodoDinamico(Ejemplo: var) -> void {
         Ejemplo.metodoEstatico();  // EXITO.
         metodoEstatico() // EXITO.
         (var.metodoEstatico()) // EXITO.
   }
}
```

2.2 Errores

En la etapa de diseño se previó que una serie de errores distintos pueden aparecer durante el análisis semántico.

Para la etapa de análisis semántico de **declaraciones** se previó que pueden aparecer

distintos errores de chequeo semántico durante el análisis y construcción de la TS y durante la posterior etapa de consolidación. Cada tipo de error se lista a continuación:

- *Constructor mal formado*: error generado al encontrar una declaración de un método de clase cuyo identificador es “create”. El error sugiere la sintaxis apropiada para declarar el constructor, ya que es ligeramente diferente de la sintaxis normal de declaración de métodos.
- *Múltiples constructores*: error generado al encontrar dos constructores declarados en una misma clase, ya que el lenguaje soporta un único constructor por clase.
- *Entidad duplicada*: error generado al declarar dos clases o dos métodos de una misma clase o dos argumentos/variables de un mismo método con el mismo identificador.
- *Herencia base ilegal*: error generado al hallar una clase que herede de cualquiera de las clases predefinidas de TinyRust+, estas son: Str, Char, I32, Bool y Array.
- *Herencia circular*: error generado durante la etapa de consolidación si una clase hereda directa o indirectamente de sí misma.
- *Clase inexistente*: error generado durante la etapa de consolidación si una clase hereda de otra clase no declarada o si un atributo, argumento de método, retorno de método o variable de método son de una clase no declarada.
- *Error de consolidación*: todo error generado durante la etapa de consolidación que no se haya mencionado previamente se cataloga en este grupo. El error incluye información explicativa para el usuario sobre el escenario en que ocurrió. Algunos escenarios posibles de este error son si se redefinió un método estático o atributo de una superclase o se redefinió un método de una superclase con distinta firma.

Para la etapa de análisis semántico de **sentencias** se previó que pueden aparecer errores durante la etapa de validación del AST, pero no durante su construcción, dado que la sintaxis y las declaraciones ya están validadas para cuando se construyen los nodos del AST. Los distintos errores de validación se listan a continuación:

- *Expresión de tipo incorrecto*: error generado al recibir una expresión que no es del tipo esperado ni un subtipo para el contexto en que se está usando. Por ejemplo, si en la condición de un if se utiliza una expresión no booleana, si se utilizan operandos de tipo string al realizar una operación de resta o similares, si se retorna en un método una expresión que no coincide con el tipo de retorno del método o si se asigna en una variable un valor de un tipo distinto al declarado para la variable
- *Entidad inaccesible*: error generado al intentar acceder a una variable, método o atributo que no están definidos para la clase actual o no son visibles. Este caso se puede dar al intentar acceder variables del método actual, al intentar acceder atributos (privados o públicos) y métodos de la clase actual, al invocar métodos estáticos, en los accesos encadenados tanto de métodos como atributos, al acceder la referencia self dentro de un método estático o al acceder atributos de instancia en un método estático.
- *Invocación inválida de método*: error generado cuando la invocación de un método no respeta la definición formal del método. Ocurre al invocar de forma dinámica un

método estático de una clase o viceversa, al invocar al método create explícitamente y al invocar el método con una cantidad incorrecta de argumentos o si sus tipos no coinciden con la definición formal.

- *Invocación inválida de arreglo*: error generado al intentar acceder un índice de un atributo de una clase cuando el atributo no es de tipo arreglo.

2.3 Esquema de traducción

La cátedra facilitó en el manual de TinyRust+ una versión de su gramática en formato EBNF. Para hacer uso de la gramática para la implementación del analizador sintáctico, se aplicaron una serie de algoritmos para transformar la gramática, sin alterar el conjunto de cadenas generados por la gramática. Los algoritmos aplicados fueron: transformación a formato BNF, eliminación de reglas lambda, eliminación de recursividad a izquierda y factorización.

Para la implementación del analizador semántico de declaraciones y sentencias, la gramática previamente tratada se extendió con atributos y acciones semánticas. A continuación se encuentra el esquema de traducción.

Como referencia, las acciones semánticas de color verde o azul están relacionadas con la TS, mientras que las acciones anaranjadas o rojas se relacionan al AST. Las acciones de color verde crean o agregan entidades a la tabla de símbolos, mientras que las anotaciones azules unicamente sintetizan o establecen atributos que serán utilizados para la construcción de la TS. Las acciones rojas crean nodos hoja del AST, mientras que las anotaciones anaranjadas sintetizan nodos o modifican el AST.

```
Start ::= Clases Main {ast.setClases(Clases.nodo)}
Clases(1) ::= Clase Clases(2) {Clases1.nodo = Clases2.nodo.add(Clase.nodo)}
    | λ  {Clases1.nodo = new TreeList<Clase>()}

Main ::=  "fn" "main" "(" ")" Bloque-Método {ts.addMain();
ast.addMain(BloqueMetodo.nodo);}

Clase ::= "class" "idClase" ClaseHerenciaOp {currClase = ts.addClase(idClase.lex);
Clase.nodo = new Clase(nombre: idClase.lex, metodos: CHO.nodo);}

Herencia ::= ":" "idClase" {currClase.heredaDe = idClase.lex;}

ClaseHerenciaOp ::= Herencia "{" Miembros "}" {CHO.nodo = Miembros.nodo}
    | "{" Miembros "}" {CHO.nodo = Miembros.nodo}

Miembros(1) ::= Miembro Miembros(2) {Miembros1.nodo =
Miembros2.nodo.addIfNotNull(Miembro.nodo)}
    | λ {Miembros1.nodo = new TreeList<Metodo>()}

Miembro ::= Atributo {Miembro.nodo = null}
    | Constructor {Miembro.nodo = Constructor.nodo}
    | Método {Miembro.nodo = Metodo.nodo}
```

Atributo ::= Tipo ":" Lista-Declaración-Variables ";" {LDV.tipo = Tipo.tipo}
| Visibilidad Tipo ":" Lista-Declaración-Variables ";" {LDV.tipo = Tipo.tipo; LDV.public = Visibilidad.public}

Constructor ::= "create" Argumentos-Formales Bloque-Método {currMetodo = ts.addConstructorClase(currClase); Constructor.nodo = new Metodo(nombre: “create”, bloque: BloqueMetodo.nodo);}
Método ::= "fn" "idMétodoVariable" Argumentos-Formales "->" Tipo-Método Bloque-Método {currMetodo = ts.addMetodoClase(currClase, new Metodo(nombre: idMetodoVariable.lex, retornoTipo: TipoMetodo.tipo); Metodo.nodo = new Metodo(nombre: idMetodoVariable.lex, bloque: BloqueMetodo.nodo);}
| Forma-Método "fn" "idMétodoVariable" Argumentos-Formales "->" Tipo-Método Bloque-Método {currMetodo = ts.addMetodoClase(currClase, new Metodo(nombre: idMetodoVariable.lex, retornoTipo: TipoMetodo.tipo, static: FormaMetodo.static); Metodo.nodo = new Metodo(nombre: idMetodoVariable.lex, bloque: BloqueMetodo.nodo);}

Argumentos-Formales ::= "(" ")"
| "(" Lista-Argumentos-Formales ")"
Lista-Argumentos-Formales ::= Argumento-Formal "," Lista-Argumentos-Formales
| Argumento-Formal

Argumento-Formal ::= Tipo ":" "idMétodoVariable" {currMetodo.addArgumento(nombre: idMetodoVariable.lex, tipo: Tipo.tipo)}

Forma-Método ::= "static" {FormaMetodo.static = true}

Visibilidad ::= "pub" {Visibilidad.public = true}

Tipo-Método ::= Tipo {TipoMetodo.tipo = Tipo.tipo}
| "void" {TipoMetodo.tipo = new Void()}

Tipo ::= Tipo-Primitivo {Tipo.tipo = TipoPrimitivo.tipo}
| Tipo-Referencia {Tipo.tipo = TipoReferencia.tipo}
| Tipo-Array {Tipo.tipo = TipoArray.tipo}

Tipo-Primitivo ::= "Bool" {TipoPrimitivo.tipo = new Bool()}
| "I32" {TipoPrimitivo.tipo = new I32()}
| "Str" {TipoPrimitivo.tipo = new Str()}
| "Char" {TipoPrimitivo.tipo = new Char()}

Tipo-Referencia ::= "idClase" {TipoReferencia.tipo = new TipoClase(nombre:idClase.lex)}

Tipo-Array ::= "Array" Tipo-Primitivo {TipoArray.tipo = new Array(tipo: TipoPrimitivo.tipo)}

Lista-Declaración-Variables(1) ::= "idMétodoVariable" {ts.addVariable(new Var(name: idMetodoVariable.lex, tipo: LDV1.tipo, public: LDV1.public))}
| "idMétodoVariable" "," Lista-Declaración-Variables(2) {ts.addVariable(new Var(name: idMetodoVariable.lex, tipo: LDV1.tipo, public: LDV1.public)); LDV2.tipo =

LDV1.tipo; LDV2.public = LDV1.public;}

Bloque-Método ::= "{" Decl-Var-LocalesN Sentencias "}" {BM.nodo = new Bloque(Sentencias.nodo)}

Decl-Var-LocalesN := Decl-Var-Locales Decl-Var-LocalesN | λ

Decl-Var-Locales ::= Tipo ":" Lista-Declaración-Variables ";" {LDV.tipo = Tipo.tipo}

Sentencias(1) ::= Sentencia Sentencias(2) {Sentencias(1).nodo = Sentencias2.nodo.add(Sentencia.nodo)}
| λ {Sentencias(1).nodo = new TreeList<Sentencias>()}

Sentencia(1) ::= ";" {Sentencia1.nodo = null}
| Asignación ";" {Sentencia1.nodo = Asignacion.nodo}
| Sentencia-Simple ";" {Sentencia1.nodo = SS.nodo}
| "if" "(" Expresión ")" Sentencia(2) ElseOp {Sentencia1.nodo = new IfElse(cond: Exp.nodo, bloque: Sentencia2.nodo, elseBloque: ElseOp.nodo)}
| "while" "(" Expresión ")" Sentencia(2) {Sentencia1.nodo = new While(cond: Expresion.nodo, bloque: Sentencia2.nodo)}
| Bloque {Sentencia1.nodo = Bloque.nodo}
| "return" Expresión ";" {Sentencia1.nodo = new Return(valor: Expresion.nodo)}
| "return" ";" {Sentencia1.nodo = new Return(valor: new Void())}

ElseOp ::= "else" Sentencia {ElseOp.nodo = Sentencia.nodo}
| λ {ElseOp.nodo = null}

Bloque ::= "{" Sentencias "}" {Bloque.nodo = new Bloque(Sentencias.nodo)}

Asignación ::= Asignación-Variable-Simple "=" Expresión {Asignacion.nodo = new Asignacion(izq: AVS.nodo, der: Exp.nodo)}
| Asignación-Self-Simple "=" Expresión {Asignacion.nodo = new Asignacion(izq: ASS.nodo, der: Exp.nodo)}

Asignación-Variable-Simple ::= "idMétodoVariable" "[" Expresión "]" {AVS.nodo = new AccesoArray(var: new Var(idMetodoVar.lex), indice: Expresion.nodo)}
| "idMétodoVariable" Encadenado-SimpleN {AVS.nodo = new AccesoVar(var: new Var(idMetodoVar.lex), cadena: ESN.nodo)}

Encadenado-SimpleN(1) ::= Encadenado-Simple Encadenado-SimpleN(2) {ESN1.nodo = new AccesoAttrEncadenado(var: ES.nodo, cadena: ESN2.nodo)}
| λ {ESN1.nodo = null}

Asignación-Self-Simple ::= "self" Encadenado-SimpleN {ASS.nodo = new AccesoVar(var: new Var(“self”), cadena: ESN.nodo)}

Encadenado-Simple ::= "." "idMétodoVariable" {ES.nodo = new Var(idMetodoVar.lex)}

Sentencia-Simple ::= "(" Expresión ")" {SS.nodo = new SentSimple(Exp.nodo)}

Expresión ::= ExpOr {Expresion.nodo = ExpOr.nodo}

```
ExpOr ::= ExpAnd ExpOrP {if (ExpOrP.nodo != null) {
      ExpOr.nodo = ExpOrP.nodo.setIzq(ExpAnd.nodo);
      } else {ExpOr.nodo = ExpAnd.nodo;}}

ExpOrP(1) ::= "||" ExpAnd ExpOrP(2) {if (ExpOrP2.nodo != null) {
      ExpOrP2.setIzq(ExpAnd.nodo);
      ExpOrP1.nodo = new OpBinaria(op: "||", derecha: ExpOrP2.nodo);
      } else {ExpOrP1.nodo = new OpBinaria(op: "||", derecha: ExpAnd.nodo);}}
   | λ {ExpOrP.nodo = null}

ExpAnd ::= ExpIgual ExpAndP {if (ExpAndP.nodo != null) {
      ExpAnd.nodo = ExpAndP.nodo.setIzq(ExpIgual.nodo);
      } else {ExpAnd.nodo = ExpIgual.nodo;}}

ExpAndP(1) ::= "&&" ExpIgual ExpAndP(2) {if (ExpAndP2.nodo != null) {
      ExpAndP2.setIzq(ExpIgual.nodo);
      ExpAndP1.nodo = new OpBinaria(op: "&&", derecha: ExpAndP2.nodo);
      } else {ExpAndP1.nodo = new OpBinaria(op: "&&", derecha: ExpIgual.nodo);}}
   | λ {ExpIgualP.nodo = null}

ExpIgual ::= ExpCompuesta ExpIgualP  {if (ExpIgualP.nodo != null) {
      ExpIgual.nodo = ExpIgualP.nodo.setIzq(ExpCompuesta.nodo);
      } else {ExpIgual.nodo = ExpCompuesta.nodo;}}


ExpIgualP(1) ::= OpIgual ExpCompuesta ExpIgualP(2) {if (ExpIgualP2.nodo != null) {
      ExpIgualP2.setIzq(ExpComp.nodo);
      ExpIgualP1.nodo = new OpBinaria(op: OpIgual.lex, derecha: ExpIgualP2.nodo);
      } else {ExpIgualP1.nodo = new OpBinaria(op: OpIgual.lex, der: ExpComp.nodo);}}
   | λ {ExpIgualP.nodo = null}

ExpCompuesta ::= ExpAdd(1) OpCompuesto ExpAdd(2) {ExpComp.nodo = new
OpBinaria(op: OpComp.lex, izq: ExpAdd1.nodo, der: ExpAdd2.nodo)}
   | ExpAdd {ExpComp.nodo = ExpAdd.nodo}

ExpAdd ::= ExpMul ExpAddP  {if (ExpAddP.nodo != null) {
      ExpAdd.nodo = ExpAddP.nodo.setIzq(ExpMul.nodo);
      } else {ExpAdd.nodo = ExpMul.nodo;}}

ExpAddP(1) ::= OpAdd ExpMul ExpAddP(2)  {if (ExpAddP2.nodo != null) {
      ExpAddP2.setIzq(ExpMul.nodo);
      ExpAddP1.nodo = new OpBinaria(op: OpAdd.lex, derecha: ExpAddP2.nodo);
      } else {ExpAddP1.nodo = new OpBinaria(op: OpAdd.lex, der: ExpMul.nodo);}}
   | λ {ExpAddP.nodo = null}

ExpMul ::= ExpUn ExpMulP  {if (ExpMulP.nodo != null) {
ExpMul.nodo = ExpMulP.nodo.setIzq(ExpUn.nodo); else {ExpMul.nodo = ExpUn.nodo;}}

ExpMulP(1) ::= OpMul ExpUn ExpMulP(2) {if (ExpMulP2.nodo != null) {
      ExpMulP2.setIzq(ExpUn.nodo);
```

```
      ExpMulP1.nodo = new OpBinaria(op: OpMul.lex, derecha: ExpMulP2.nodo);
      } else {ExpMulP1.nodo = new OpBinaria(op: OpMul.lex, der: ExpUn.nodo);}}
    | λ {ExpMulP.nodo = null}

ExpUn(1) ::= OpUnario ExpUn(2) {ExpUn1.nodo = new ExpUnaria(op: OpUnario.lex,
exp: ExpUn2.nodo}
    | Operando {ExpUn1.nodo = Operando.nodo}

OpIgual ::= "=="
    | "!="

OpCompuesto ::= "<"
    | ">"
    | "<="
    | ">="

OpAdd ::= "+"
    | "-"

OpUnario ::= "+"
    | "-"
    | "!"
OpMul ::= "*"
    | "/"
    | "%"

Operando ::= Literal {Op.nodo = new Literal(Literal.lex)}
    | Primario EncadenadoOp {Op.nodo = new ExpEncadenada(exp: Primario.nodo,
cadena: EncadenadoOp.nodo)}}

Literal ::= "nil"
    | "true"
    | "false"
    | "intLiteral"
    | "stringLiteral"
    | "charLiteral"

Primario ::= ExpresionParentizada  {Primario.nodo = ExpresionParentizada.nodo}
    | AccesoSelf {Primario.nodo = AccesoSelf.nodo}
    | AccesoVar {Primario.nodo = AccesoVar.nodo}
    | Llamada-Método  {Primario.nodo = LlamadaMetodo.nodo}
    | Llamada-Método-Estático {Primario.nodo = LlamadaMetodoEstatico.nodo}
    | Llamada-Constructor {Primario.nodo = LlamadaConstructor.nodo}

ExpresionParentizada ::= "(" Expresión ")" EncadenadoOp {ExpParent.nodo = new
ExpEncadenada(exp: Expresion.nodo, cadena: EncadenadoOp.nodo}

Lista-Expresiones(1) ::= Expresión {LE1.nodo = new TreeList<Expresion>()}
    | Expresión "," Lista-Expresiones(2) {LE1.nodo = LE2.nodo.add(Exp.nodo)}

AccesoSelf ::= "self" EncadenadoOp {AccesoSelf.nodo = new AccesoVar(var: new
```

```
Var("self"), cadena: EncadenadoOp.nodo)}

AccesoVar ::= "idMétodoVariable" EncadenadoOp {AccesoVar.nodo = new
AccesoVar(var: new Var(idMetodoVariable.lex), cadena: EncadenadoOp.nodo)}
    | "idMétodoVariable"  "[" Expresión "]" {AccesoVar.nodo = new AccesoArray(var: new
Var(idMetodoVariable.lex), indice: Expresion.nodo)}

Llamada-Método ::= "idMétodoVariable" Argumentos-Actuales EncadenadoOp {LM.nodo
= new LlamadaMetodo(nombre: new Var(idMetodoVariable.lex), args:
ArgumentosActuales.nodo, acceso: EncadenadoOp.nodo)}

Llamada-Método-Estático ::= "idClase" "." Llamada-Método EncadenadoOp {LM.nodo =
new LlamadaMetodoEstatico(clase: idClase.lex, llamado: LlamadaMetodo.nodo,
acceso: EncadenadoOp.nodo)}

Llamada-Constructor ::= "new" "idClase" Argumentos-Actuales EncadenadoOp
{LM.nodo = new LlamadaConstructor(args: ArgumentosActuales.nodo, acceso:
EncadenadoOp.nodo)}
    | "new" Tipo-Primitivo "[" Expresión "]" {LM.nodo = new
LlamadaConstructorArray(tipo: TipoPrimitivo.nodo, tamaño: Expresion.nodo)}

Argumentos-Actuales ::= "(" ")" {ArgAct.nodo = new TreeList<Expresion>()}
    | "(" Lista-Expresiones ")" {ArgAct.nodo = ListaExp.nodo}

Encadenado ::= "." Llamada-Método-Encadenado {Encadenado.nodo = LME.nodo}
    | "." Acceso-Variable-Encadenado {Encadenado.nodo = AVE.nodo}

Llamada-Método-Encadenado ::= "idMétodoVariable" Argumentos-Actuales
EncadenadoOp {LME.nodo = new LlamadaMetodoCadena(nombre:  new
Var(idMetodoVariable.lex), args: ArgumentosActuales.nodo, cadena:
EncadenadoOp.nodo)}

Acceso-Variable-Encadenado ::= "idMétodoVariable" EncadenadoOp {AccesoVar.nodo =
new AccesoVarCadena(var: new Var(idMetodoVariable.lex), cadena:
EncadenadoOp.nodo)}
    | "idMétodoVariable" "[" Expresión "]" {AccesoVar.nodo = new AccesoArray(var: new
Var(idMetodoVariable.lex), tamaño: Exp.nodo)}

EncadenadoOp ::= Encadenado
    | λ {EncadenadoOp.nodo = Encadenado.nodo}
```

2.4 Chequeos semánticos

En la sección 2.2 ya se explicaron brevemente los tipos de error que pueden ocurrir durante el análisis semántico. En esta sección se explicará con mayor detalle cada validación.

Los chequeos semánticos de declaraciones ocurren en dos momentos: durante la construcción de la tabla de símbolos y durante la consolidación de la tabla. Durante la

construcción de la tabla de símbolos se realizan los siguientes chequeos:

1. Al guardar una **clase** en la TS, se valida que no se haya guardado previamente una clase con el mismo nombre.
2. Al guardar una **clase** en la TS, se valida que no herede de una clase base o predefinida de TinyRust+.
3. Al guardar el **constructor** de una clase en la TS, se valida que no haya guardado previamente otro constructor para esa misma clase, dado que TinyRust+ no soporta múltiples constructores por clase.
4. Al guardar un **atributo** de una clase en la TS, se valida que no se haya guardado previamente otro atributo en esa misma clase con el mismo nombre.
5. Al guardar un **atributo** de una clase en la TS, se valida que su nombre no sea “self”, ya que colisionará con la referencia a la instancia de la clase.
6. Al guardar un **método** de una clase en la TS, se valida que no se haya guardado previamente otro método en esa misma clase con el mismo nombre.
7. Al guardar un **método** de una clase en la TS, se valida que su nombre no sea “self”, ya que colisionará con la referencia a la instancia de la clase.
8. Al guardar un **método** de una clase en la TS, se valida que su nombre no sea “create”, ya que en este caso debería declararse el método con la sintaxis del constructor (que no incluye el tipo de retorno ni la palabra reservada “fn”).
9. Al guardar un **argumento** de un método de una clase en la TS, se valida que no se haya guardado previamente otro argumento en ese mismo método con el mismo nombre.
10. Al guardar un **argumento** de un método de una clase en la TS, se valida que su nombre no sea “self”, ya que colisionará con la referencia a la instancia de la clase.
11. Al guardar una **variable** del bloque de un método en la TS, se valida que no se haya guardado previamente otra variable o argumento en ese mismo método con el mismo nombre.
12. Al guardar una **variable** del bloque de un método en la TS, se valida que su nombre no sea “self”, ya que colisionará con la referencia a la instancia de la clase.

Durante la etapa de consolidación de la tabla de símbolos se realizan los siguientes chequeos semánticos:

1. Para cada **clase** guardada en la TS se valida que su superclase esté definida en la tabla (a excepción de las clases Object y main, que no poseen superclase).
2. Para cada **clase** guardada en la TS se valida que no posea herencia circular, es decir, que no herede directa o indirectamente de sí misma.
3. Para cada **atributo** de una clase, se valida que su tipo coincida con una clase definida en la TS.
4. Para cada **atributo** de una superclase, se valida que la subclase no defina un atributo con el mismo nombre, ya que TinyRust+ no soporta redefinición de atributos de ningún tipo.
5. Para cada **método estático** de una superclase, se valida que la subclase no defina un método con el mismo nombre, ya que TinyRust+ no soporta redefinición de

métodos estáticos.

6. Para cada **método no estático** de una superclase, se valida que la subclase no defina un método con el mismo nombre y distinta firma o modificador, ya que TinyRust+ no soporta redefinición de métodos con distinta firma. En este caso se valida que coincidan el tipo de retorno y la cantidad, orden y tipo de argumentos formales para que el método se considere correctamente redefinido.
7. Para cada **argumento** y **tipo de retorno** de un método se valida que su tipo coincida con una clase definida en la TS.
8. Para cada **variable** en el bloque de un método se valida que su tipo coincida con una clase definida en la TS.

Los chequeos semánticos de sentencias ocurren luego de que se hayan construido por completo el AST y la tabla de símbolos esté completa y consolidada. Las validaciones de sentencias que se efectúan sobre el AST son las siguientes:

1. Para cada **método** se valida que el tipo retornado por el bloque coincida con el tipo de retorno formal del método.
2. Para cada **método estático** se valida que no se acceda a la referencia self ni a atributos o métodos de instancia.
3. Para cada **bloque de método**, se verifica que todas sus ramas o bloques condicionales anidados retornen un tipo de dato que coincida o sea una subclase del tipo de retorno del método.
4. Para cada **sentencia de asignación** se valida que el tipo de la expresión del lado derecho sea el mismo o una subclase del tipo de la expresión del lado izquierdo.
5. Para cada **sentencia if** se valida que el tipo de la expresión condicional sea booleano.
6. Para cada **sentencia while** se valida que el tipo de la expresión condicional sea booleano.
7. Para cada **sentencia return** se valida que el tipo de la expresión retornada coincida con el tipo de retorno formal del método o sea una subclase de esta.
8. Para cada **expresión** se valida que el tipo retornado sea el esperado para el contexto en el cuál es usada.
9. Para cada **expresión binaria** se valida que ambos operandos sean del mismo tipo, En el caso de las operaciones aritméticas, se valida que ambos operandos sean de tipo I32. En las operaciones lógicas se valida los operandos sean de tipo booleano. Mientras que en las operaciones *!=* y *==* no se exige ningún tipo particular.
10. Para cada **expresión unaria** se valida que el operando sea del tipo correcto para el operador. En caso de los operadores de signo se espera que el operando sea de tipo I32, mientras que para el operador de negación se espera que el operando sea de tipo booleano.
11. Para cada **acceso a variable** se valida que exista una variable, argumento o atributo visible en el contexto actual con tal identificador. En el caso de los atributos, se valida que el atributo no sea privado heredado, ya que no sería visible en tal caso.
12. Para cada **acceso a método dinámico** se valida que la clase actual posea un

método con tal nombre y que la cantidad, tipo y orden de argumentos coincida con la definición formal.

13. Para cada **acceso a método estático** se valida que la clase actual posea un método estático con tal nombre y que la cantidad, tipo y orden de argumentos coincida con la definición formal.
14. Para cada **acceso a atributo encadenado** se valida que la clase accedida posea un atributo visible con tal nombre. Dentro de la misma clase, son visibles todos los atributos excepto los privados heredados. Fuera de la clase, son visibles únicamente los atributos públicos.
15. Para cada **acceso a método encadenado** se valida que la clase accedida posea un método con tal nombre y que la cantidad, tipo y orden de argumentos coincida con la definición formal.
16. Para cada **acceso a elemento de un arreglo (simple o encadenado)** se valida que la entidad accedida sea de tipo arreglo y que la expresión del índice sea de tipo I32.
17. Para cada **llamada a constructor** se valida que la cantidad, tipo y orden de argumentos coincida con la definición formal.
18. Para cada **llamada a constructor de arreglo** se valida que el tamaño del arreglo sea un literal entero. Para simplificar el análisis, no se aceptan expresiones para asignar el tamaño de un arreglo, es decir, el tamaño de los arreglos debe conocerse estáticamente en tiempo de compilación.

3. Resultados

3.1 Estructura general

El repositorio del proyecto se encuentra estructurado en una jerarquía de directorios para facilitar su legibilidad. En la **Figura 1** puede apreciarse esta jerarquía.

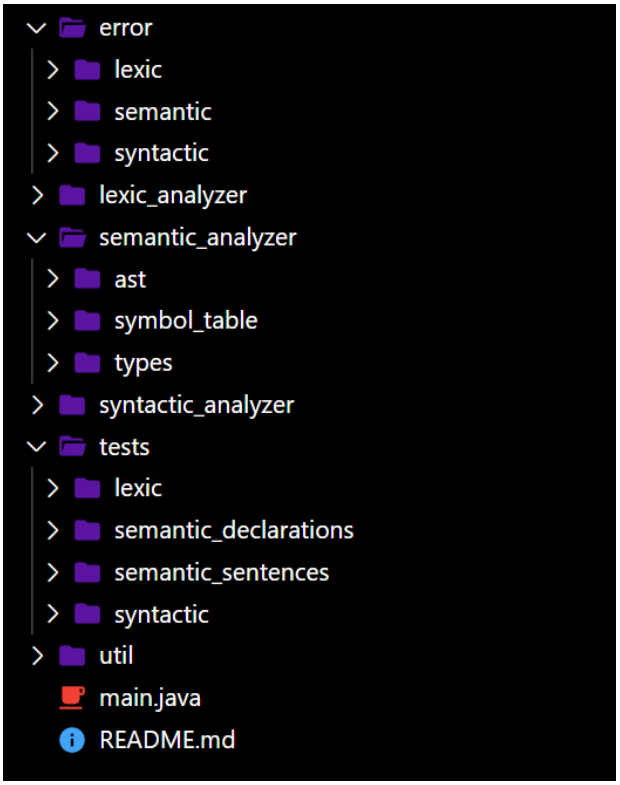


Figura 1. Jerarquía de carpetas

En el directorio raíz se encuentran los siguientes archivos y directorios:

1. Archivo *main.java*: contiene la clase TinyRustCompiler, el punto de entrada del compilador.

2. Carpeta *util*: contiene clases que proveen algún tipo de asistencia mediante métodos. Por ejemplo: la clase Logger (provee métodos necesarios para escribir mensajes en el archivo de salida) y la clase Json (provee métodos para generación de archivos en formato JSON[2]).

3. Carpeta *tests*: contiene los archivos *.rs* de prueba utilizados para probar el compilador. Contiene una carpeta por cada etapa de análisis con archivos de prueba para esa etapa. Cada archivo de prueba incluye el output esperado al analizarlo.

4. Carpeta *lexic_analyzer:* contiene todas las clases relacionadas al analizador léxico del compilador. Incluye al analizador léxico, un ejecutor y las clases *Token* y *ReservedWords*.

5. Carpeta *syntactic_analyzer*: contiene todas las clases relacionadas al analizador sintáctico del compilador. Incluye al analizador sintáctico y un ejecutor.

6. Carpeta *semantic_analyzer*: incluye una subcarpeta para clases relacionadas con el AST, otra para la TS y una conteniendo tipos compartidos por ambas partes. En el caso de la TS se incluyen clases para cada tipo de entidad de la tabla y la clase que representa a la tabla de símbolos propiamente. En el caso del AST se incluyen clases para cada tipo de nodo del árbol y la clase que representa el nodo raíz del

árbol. En la última carpeta se incluye una clase para cada tipo de dato de TinyRust+, que luego se utilizan tanto en la TS como en el AST.

7. Carpeta *error*: contiene una clase para cada posible error que pueda ser arrojado al momento de ejecutar el compilador de TinyRust+. Contiene una carpeta por cada etapa de análisis, donde se encuentra una clase para cada error que pueda aparecer en esa etapa.

3.2 **Implementación**

En esta sección se ahondará en la implementación del analizador semántico de sentencias de TinyRust+, explicando el rol, funcionamiento y métodos de cada una de las clases que lo componen. La **Figura 2** muestra el diagrama de clases UML de la tabla de símbolos de TinyRust+. Mientras que la **Figura 3** muestra un diagrama de las clases que componen al árbol sintáctico abstracto. Este último diagrama contiene los atributos y relaciones relevantes para el entendimiento de la arquitectura, dado que de incluir en el diagrama absolutamente todas las clases de otros paquetes, atributos y todas las relaciones de dependencia dificultaría su entendimiento dada su complejidad. La división en dos diagramas es meramente para una mejor visualización, dado que contienen una gran cantidad de clases, pero ambos diagramas pertenecen a distintas partes del mismo proyecto. De hecho algunas clases, como *SyntacticAnalyzer*, *Type* y *SymbolTable* aparecen en ambos diagramas, se podría decir que son el punto de conexión entre estos.

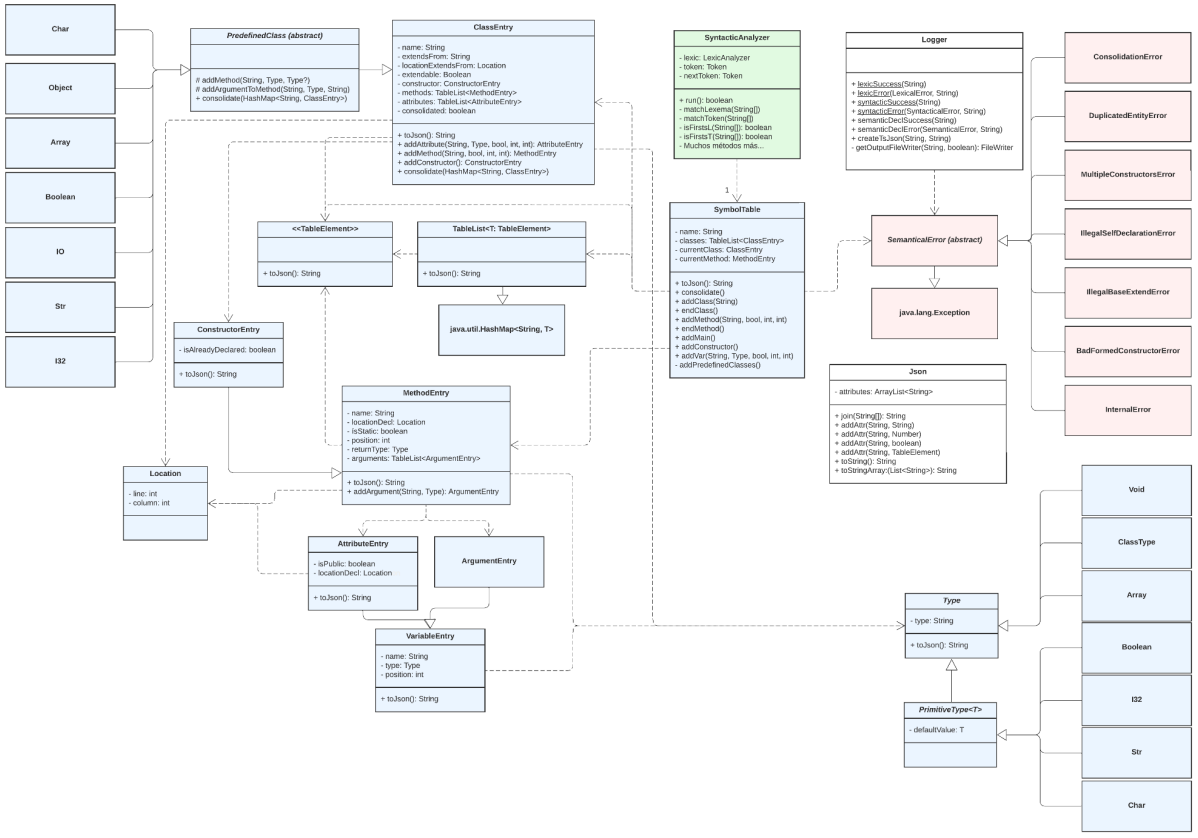


Figura 2. Diagrama de clases UML de la tabla de símbolos de TinyRust+.

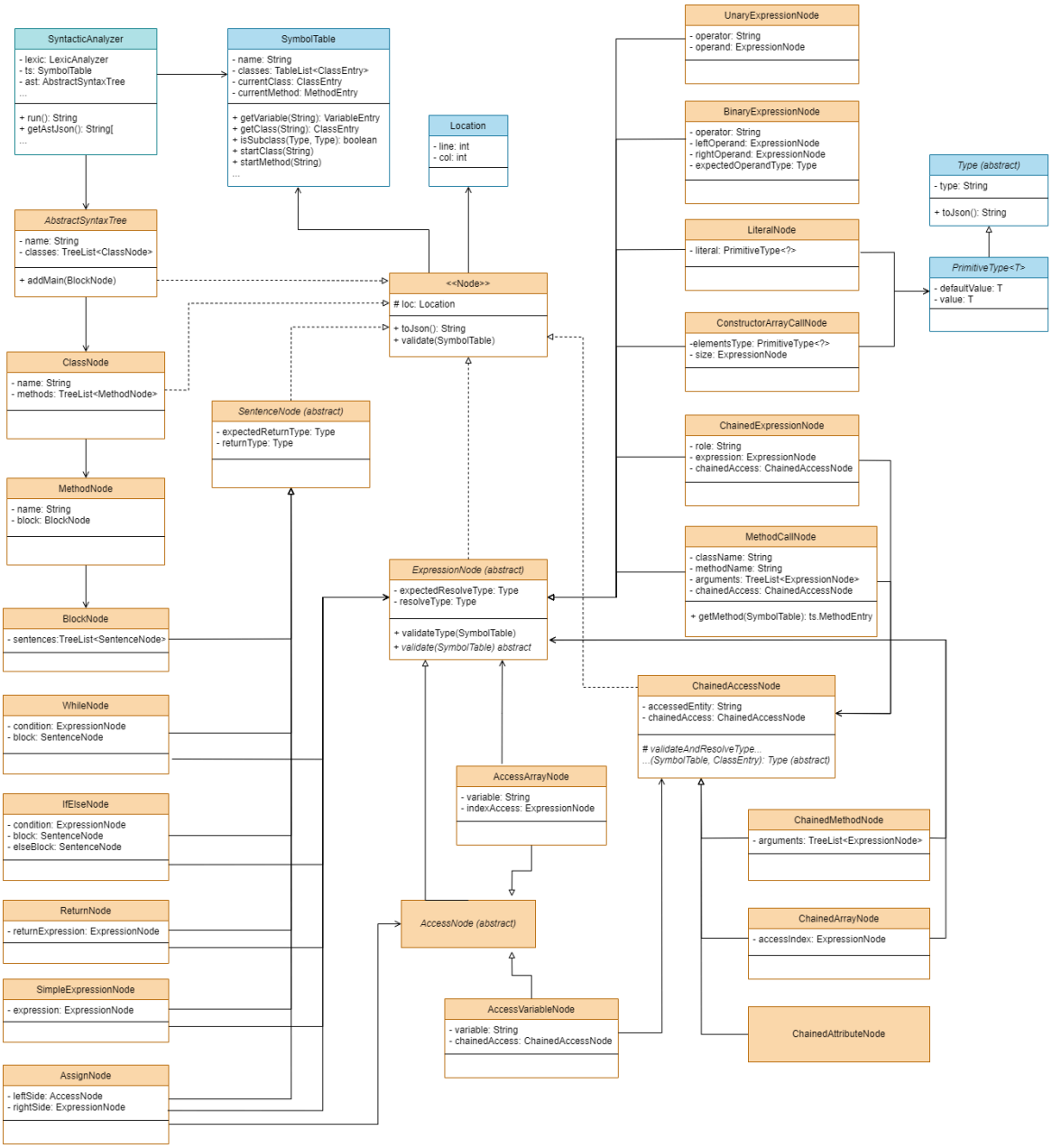


Figura 3. Diagrama de clases del AST de TinyRust+.

3.2.1 **TinyRustCompiler**

Esta es la clase raíz del compilador. Recibe los argumentos de la línea de comandos para los archivos de entrada y salida y se encarga de guardarlos. En el método main de esta clase se instancia la clase *Executor* y se invoca su método *run* para comenzar la ejecución del compilador.

3.2.2 **Executor (syntactic_analyzer/Executor)**

Esta clase consta de un único método llamado *run*. Al invocar este método, se crea una instancia de la clase *SyntacticAnalyzer* que recibe como parámetro la ruta del archivo de entrada. A su vez la ruta de entrada se utiliza para construir el nombre de la ruta de los archivos de salida de la TS y el AST cambiando únicamente la extensión del archivo, pero conservando el nombre.

Una vez instanciado el analizador, el ejecutor llamará al método *run* del *SyntacticcAnalyzer*, el cual realiza el análisis sintáctico y construye la tabla de símbolos y el AST. Cuando este método retorna, la tabla de símbolos se encuentra ya unificada y el AST también se encuentra completo y validado. En caso de un resultado positivo, un mensaje de éxito se muestra por pantalla y se escriben dos archivos: una representación en formato JSON de la tabla de símbolos se escribe en un archivo con extensión *.ts.json* y una representación JSON del árbol sintáctico abstracto se guarda en un archivo con extensión *.ast.json*. Si el archivo no fuera sintáctica o semánticamente correcto, *Executor* intercepta la excepción lanzada por *SyntacticAnalyzer* y en la salida muestra línea y columna donde se presentó el error, incluyendo en la descripción si se trata de un error léxico, sintáctico o semántico. Si el archivo contuviera declaraciones o sentencias inválidas semánticamente para TinyRust+, el método run lanzará una excepción de tipo *SemanticalError*, que será interceptada y escrita en el medio de salida, incluyendo línea, columna y descripción del error.

3.2.3 **SyntacticAnalyzer**

Esta clase es la encargada de todo el proceso de análisis sintáctico y construcción de las estructuras de análisis semántico. El análisis se hace de forma descendente y predictiva, mientras que la implementación en esta clase es recursiva.

En su constructor se inicializan: una instancia de *LexicAnalyzer*, una instancia de *SymbolTable* y una instancia de *AbstractSyntaxTree*.

SyntacticAnalyzer provee el método público *run*. Este método inicializa la tabla de símbolos y comienza el proceso de análisis sintáctico del archivo utilizando la gramática de TinyRust+. En este informe no se ahondará en los métodos y el flujo mediante el cuál se realiza el análisis sintáctico, sino que el enfoque está puesto en cómo se construyen las estructuras propias del análisis semántico durante la fase de análisis sintáctico.

La clase *SyntacticAnalyzer* posee una gran cantidad de métodos que se corresponden cada uno con una regla (homónima) de la gramática final de TinyRust+. Cada método implementa no solo la lógica de *matcheo* sintáctico de la regla correspondiente, sino también las acciones semánticas de la regla. El método correspondiente a una regla se ejecutará por completo sin problemas si la secuencia de tokens encontrados es derivable mediante esa regla de la gramática y no infringe ninguna regla semántica al agregar entidades a la TS. Mientras *matchea* tokens, la regla puede agregar entidades a la tabla de símbolos, modificar el estado de la tabla de símbolos o construir y componer nodos del árbol sintáctico abstracto. Cuando el método termina de ejecutar, posiblemente retorne un nodo del AST o una entidad para la TS, dependiendo de la regla. En caso de que la entrada no sea correcta sintácticamente o se agregue una entidad inválida a la TS, la regla arrojará una excepción y no completará su ejecución.

Cuando el archivo de entrada se consumió por completo, el analizador sintáctico

inicia la “segunda pasada” de análisis. Primero invoca al método *consolidate* de la tabla de símbolos y luego invoca al método *validate* del AST. Luego de una segunda pasada exitosa, el método *run* devuelve la representación JSON de la tabla de símbolos, mientras que la representación JSON del AST se puede obtener invocando al método *getAstJson* de la misma clase.

3.2.4 **SymbolTable**

Esta clase implementa las responsabilidades de la tabla de símbolos. En sus atributos guarda registro del nombre del archivo de entrada, la lista de clases definidas, y la clase y el método analizados actualmente.

Para inicializar la clase únicamente se debe invocar su constructor proporcionando el nombre del archivo. En su lista de clases incluirá por defecto todas las clases predefinidas de TinyRust+.

Provee distintos tipos de métodos:

- Métodos para la construcción de la TS como *addClass, addMain, addMethod, addConstructor* y *addVar.* Estos métodos agregan distintos tipos de entidades a la tabla de símbolos, siempre y cuando no se viole ninguna restricción semántica.
- Métodos para la consolidación de la TS como *consolidate* y *toJson.* Estos métodos se invocan cuando la tabla ya ha sido construida para realizar la segunda pasada y generar una representación JSON de la tabla.
- Métodos para facilitar datos al AST como *getVariable, getClas*s e *isSubclas*s. Mientras que las primeras dos devuelven entidades de la TS, la tercera valida si una clase es subclase de otra clase dada.

Los atributos *currentClass* y *currentMethod* guardan el estado actual de análisis de la entrada tanto durante la construcción de la TS como durante la segunda pasada de validación del AST. Durante la construcción de la TS, ambas variables facilitan la tarea de agregar métodos, atributos, argumentos y variables a las clases/métodos de la tabla, ya que todas estas entidades siempre son agregadas al método o clase actual. Durante la etapa de validación del AST, estas variables de estado guardan registro de qué clase y método se están validando actualmente. Esto facilita la tarea de obtener una variable o método de la TS para validar una expresión guardada en el AST.

3.2.5 **Entidades de la Tabla de Símbolos**

Para cada entidad de la Tabla de Símbolos, se creó una clase distinta que implementa su comportamiento.

La clase *ClassEntry* representa una clase declarada en un archivo de TinyRust+. Entre sus atributos guarda: nombre de la clase, nombre de la superclase, referencia al constructor, booleano indicando si es heredable, lista de atributos y lista de

métodos.

La clase *MethodEntry* representa un método de una clase definida en TinyRust+. Incluye distintos atributos que registran el nombre del método, su posición, tipo de retorno, lista de argumentos y un booleano indicando si es un método de instancia o de clase.

La clase *ConstructorEntry* hereda de *MethodEntry* y representa el constructor de una clase de TinyRust+. Guarda registro de si el constructor fue declarado de forma explícita o es el constructor implícito por defecto.

La clase *VariableEntry* representa una variable del lenguaje TinyRust+. En sus atributos guarda el nombre, tipo y posición de la variable.

La clase *AttributeEntry* representa un atributo de una clase de TinyRust+ y hereda de la clase *VariableEntry*. Únicamente agrega el registro de si el atributo es público o privado.

La clase *ArgumentEntry* representa un argumento formal de un método en el lenguaje TinyRust+ y hereda de la clase *VariableEntry*. No agrega ninguna funcionalidad a la superclase, su valor es meramente semántico.

La clase abstracta *PredefinedClass* representa una clase predefinida de TinyRust+. Extiende de *ClassEntry* y es a su vez heredada por las clases *Array, Bool, Char, I32, IO*, *Object* y *Str*. Todas estas clases concretas representan cada una a una clase predefinida distinta de TinyRust+ y son agregadas como entidad a la lista de clases de la tabla de símbolos durante su inicialización. Todas las clases predefinidas se inicializan con su constructor en *null* y como no heredables.

3.2.6 AbstractSyntaxTree

La clase *AbstractSyntaxTree* representa el nodo raíz del AST del archivo de entrada. Entre sus atributos guarda una lista de nodos de clase (*ClassNode*). Cuando es instanciada, inicializa esta lista y agrega un nodo para cada clase predefinida de TinyRust+ a modo de *placeholder*.

Esta clase se instancia en *SyntacticAnalyzer* antes de comenzar el análisis. Durante el análisis sintáctico, se irán generando instancias de *ClassNode* a medida que se encuentren declaraciones de clases de TinyRust+ en el archivo de entrada. Dado que el AST se construye desde las hojas hasta la raíz, cuando un *ClassNode* se agrega a la lista, esto implica que todo su subárbol ya ha sido construido, por lo cual no se está agregando un solo nodo al AST, sino una rama completa del árbol. Se incluye en la lista también un nodo de tipo *ClassNode* representando al método main, esta "clase" posee un único método llamado de igual forma que guarda todas las sentencias de main.

A la hora de validar el AST, la validación recursiva comienza invocando al

método *validate* de *AbtractSyntaxTtee* con la tabla de símbolos ya consolidada como único argumento. Este método recursivo únicamente invoca al método homónimo de cada *ClassNode* de la lista de clases. Si todos los *ClassNodes* pasan la validación, el AST completo se considera válido. El método *validate* de cualquier nodo espera recibir la tabla de símbolos ya consolidada (una instancia de *SymbolTable*) y lanza una excepción de tipo *ASTError* en caso de que el nodo o su subárbol no sean válidos semánticamente. Si *validate* completa su ejecución, implica que el nodo y todo su subárbol son válidos.

Para cada tipo de nodo distinto del AST se implementó una clase. A continuación se explica la composición y funcionamiento de los principales tipos de nodos.

3.2.7 ClassNode

Una instancia de *ClassNode* representa en el AST una clase de TinyRust+ declarada en el archivo de entrada. En sus atributos guarda únicamente el nombre de la clase y una lista *MethodNodes* representando a cada método declarado para esta clase. En el AST no se guarda registro de los atributos de cada clase, ya que esta información se encuentra en la TS. Si la clase posee la declaración de un constructor, se incluirá una entrada de método extra en la lista, con el nombre "create" y la información del constructor, de modo que sus sentencias puedan ser validadas.

Cuando se instancia un elemento de esta clase, recibe como argumentos el nombre de la clase y la lista de métodos ya construida por completo. A partir de entonces, el elemento no se modifica, únicamente se valida posteriormente.

La validación de este nodo consiste en validar cada uno de los métodos de la lista. Para esta validación, se modifica el estado de la TS actualizando *currentClass* con la entrada de clase (de la TS) correspondiente a medida que se analiza cada clase. Esto es para poder acceder en la TS a los métodos y atributos locales de la clase analizada sin que cada nodo del árbol deba saber explícitamente el nombre de la clase. Una clase se considera válida si todos sus métodos pasan la validación.

3.2.8 MethodNode

Una instancia de *MethodNode* representa en el AST un método o constructor de una clase de TinyRust+. En sus atributos guarda el nombre del método y un *BlockNode*, una referencia al bloque de sentencias del método. Tanto los argumentos del método como tipo de retorno y declaración de variables del bloque se encuentran ya guardados en la TS, por lo cual no se agregan a este nodo del AST.

Cuando se instancia un elemento de esta clase, espera recibir el nombre de la clase y el bloque de sentencias ya construído. Si el método pertenece a una clase predefinida, se agrega el flag *mocked* indicando que no se deben realizar validaciones sobre este nodo. Esto es debido a que los métodos predefinidos son

únicamente un *placeholder* y no poseen sentencias, por lo cual no suelen respetar el tipo de retorno esperado para su bloque y no pasarían la validación.

El método *validate* de MethodNode modifica el estado de la TS actualizando el valor de *currentMethod* con la entrada de la TS del método en cuestión. De este modo las variables del bloque y argumentos formales son accesibles para las validaciones de sentencias del bloque sin necesidad de que cada nodo sepa explícitamente el nombre del método. La validación del método contempla dos factores: primero, que el bloque sea válido (es decir, que cada sentencia del bloque sea válida) y segundo, que el bloque tenga un tipo de retorno que coincida con el tipo de retorno declarado para el método.

3.2.9 SentenceNode

Esta es una clase abstracta que engloba a los distintos tipos de sentencias de TinyRust+. De esta clase heredan: *BlockNode*, *AssignNode*, *SimpleExpressionNode*, *IfElseNode*, *WhileNode* y *ReturnNode*.

SentenceNode guarda entre sus atributos el tipo esperado de retorno de la sentencia y el tipo real retornado por la sentencia. Estos atributos son utilizados para validar que todas las ramas del bloque de un método retornen el tipo correcto.

La clase *BlockNode* representa un bloque de sentencias de TinyRust+. Durante la etapa de validación, este nodo invoca al método *validate* de cada una de sus sentencias y se considera válido si todas sus sentencias son válidas. Si alguna de sus sentencias tiene un tipo de retorno, entonces determina el tipo de retorno del bloque.

La clase *AssignNode* representa una asignación de una expresión a una variable. Posee dos atributos: *leftSide* de tipo *AccessNode* y *rightSide* de tipo *ExpressionNode*. En este nodo se valida que el tipo de la expresión evaluada del lado derecho coincida con el tipo del nodo accedido del lado izquierdo de la asignación.

La clase *SimpleExpressionNode* representa una sentencia simple y guarda únicamente una expresión. El método *validate* de esta sentencia se encarga de llamar al método *validate* de la expresión.

La clase *IfElseNode* representa una sentencia *if/else* de TinyRust+. Entre sus atributos guarda un *ExpressionNode* representando la condición, una *SentenceNode* representando la sentencia condicional y opcionalmente una segunda sentencia del bloque *else*. Su método *validate* corrobora que la expresión condicional sea válida y resuelva un booleano, mientras que valida recursivamente la sentencia del bloque condicional y el bloque *else* si existe. Si las sentencias de ambas ramas retornan un valor, entonces la sentencia *IfElseNode* tendrá el mismo tipo de retorno, caso contrario queda indefinido.

La clase WhileNode representa un bucle *while* de TinyRust+. En sus atributos

guarda un *ExpressionNode* representando la condición y una *SentenceNode* representando la sentencia del bucle. Su método *validate* corrobora que la expresión condicional sea válida y resuelva un booleano, mientras que valida recursivamente la sentencia del bloque condicional. Si la sentencia del bucle retorna un tipo de valor, entonces la sentencia *WhileNode* tendrá el mismo tipo de retorno, caso contrario queda indefinido.

La clase *ReturnNode* representa una sentencia return de TinyRust+. Este nodo guarda un *ExpressionNode* representando la expresión de retorno. En caso de no retornar ningún valor o retornar *nil*, la expresión es de tipo *Void().* Su método *validate* corrobora que la expresión sea válida y el tipo resuelto de la expresión coincida con el tipo de retorno esperado para el método actual. El tipo de retorno de esta sentencia es, justamente, el mismo tipo resuelto por la expresión. Este tipo es el que luego determinará el tipo de la sentencia *BlockNode* que la contiene.

3.2.10 ExpressionNode

ExpressionNode es una clase abstracta que engloba a los distintos tipos de expresiones de TinyRust+. La característica principal de este tipo de nodo es que todas sus subclases deben resolver un valor. De esta clase heredan: *BinaryExpressionNode, UnaryExpressionNode, LiteralNode, ChainedExpressionNode, AccessNode, ConstructorArrayCallNode* y *MethodCallNode.*

Esta clase abstracta guarda dos atributos de tipo *Type*: *expectedResolveType* y *resolveType*, respectivamente son el tipo que se espera que la expresión resuelva (según su contexto) y el tipo formal efectivamente resuelto por la expresión tras validarla. El método *validate* debe ser implementado por sus subclases concretas, pero *ExpressionNode* provee el método *validateType* para corroborar que el tipo de dato esperado y el resuelto coinciden. Si no hay tipo esperado o el tipo resuelto es subclase del esperado entonces la validación también pasa.

La clase *BinaryExpressionNode* representa una operación binaria ya sea lógica o aritmética. Guarda dos atributos de tipo *ExpressionNode* representando ambos operandos y un atributo de tipo *String* que guarda el operador. Cuando se instancia esta clase se debe especificar el tipo que retorna la operación y cuál es el tipo esperado para los operandos si los hay. Al invocar el método *validate* de esta clase se validan ambos operandos y se verifica que el tipo de ambos coincida con el esperado.

La clase *UnaryExpressionNode* representa una operación unaria ya sea lógica o aritmética. Guarda un atributo de tipo *ExpressionNode* representando el operando y un atributo de tipo *String* que guarda el operador. Cuando se instancia esta clase se debe especificar el tipo que retorna la operación y cuál es el tipo esperado para el operando. Al invocar el método *validate* de esta clase se valida la expresión del operando y se verifica que el tipo resuelto coincida con el esperado.

La clase *LiteralNode* representa un literal de TinyRust+ ya sea entero, cadena, caracter o booleano. Cuando se instancia espera recibir el valor del literal y su tipo. Guarda únicamente un elemento de tipo *PrimitiveType* que contiene tanto el tipo como el valor del literal. Su método *validate* únicamente valida que el tipo del literal sea el esperado para su contexto llamando a *validateType* de la superclase *ExpressionNode*. Las instancias de esta clase suelen ser las hojas del AST, por lo tanto, son de los primeros nodos en generarse.

La clase abstracta *AccessNode* representa el acceso a una variable local del contexto actual, ya sea del método o clase actual. Estas expresiones suelen formar parte del lado izquierdo de una asignación o ser el primer nodo de un encadenado. Las clases *AccessArrayNode* y *AccessVariableNode* heredan de esta clase, aunque son muy similares. La primera guarda el nombre de la variable y una expresión del índice accedido, mientras que la segunda guarda el nombre de la variable y un encadenado opcional. El método *validate* de ambas subclases es similar: se obtiene la entidad de la TS, se valida que exista y sea del tipo esperado, en caso de ser un atributo se valida que no se acceda en un contexto estático. Si es un arreglo, se valida que la expresión del índice sea de tipo I32; si es una variable común, se valida el encadenado si existe. Si se accedió un atributo se valida que sea visible en ese contexto. Si se accedió la referencia self, se valida que no se esté reasignando. Luego de resolver el tipo de retorno, se valida que este tipo coincida con el tipo esperado para su contexto.

La clase *ChainedExpressionNode* guarda una expresión de tipo *ExpressionNode* junto con un acceso encadenado de tipo *ChainedAccessNode*. Esta clase se usa para representar distintos tipos de expresiones que poseen un encadenado, como primario encadenado, expresión parentizada y llamadas encadenadas a métodos estáticos. El tipo resuelto por esta expresión es el resultado del último acceso de la cadena de encadenados. En su método *validate* simplemente se valida la expresión y se resuelve el tipo del encadenado recursivamente.

La clase *MethodCallNode* es una abstracción para distintos tipos de accesos a un método, como pueden ser acceso a un método de instancia, acceso a un método estático o llamada al constructor de una clase. Guarda el nombre del método junto con una lista de *ExpressionNodes* representando los argumentos del método y un *ChainedAccessNode* representando un encadenado opcional. El método *validate* de esta clase realiza varias validaciones: que el método exista en el contexto dado y que la cantidad y tipo de cada argumento coincida con la definición formal del método, que el encadenado sea válido y que el tipo resuelto para el método junto con el encadenado sea el esperado para su contexto. El tipo resuelto por esta expresión es igual al tipo retornado en última instancia por el encadenado.

La clase *ConstructorArrayCallNode* representa el llamado al constructor de un arreglo. Guarda un atributo de tipo *PrimitiveType* representando el tipo de dato de los elementos del arreglo y un nodo de tipo *ExpressionNode* conteniendo la

expresión del tamaño del arreglo. Su método *validate* simplemente valida que el tipo de la expresión de tamaño sea de I32 y que el arreglo sea del tipo esperado para su contexto.

3.2.11 ChainedAccessNode

ChainedAccessNode es una clase abstracta utilizada para construir accesos encadenados. Cada nodo de este tipo representa un acceso a método o atributo en una cadena de accesos y posee una referencia al siguiente nodo de la cadena. El tipo resuelto por una cadena de accesos es igual al tipo resuelto por el último nodo de la cadena.

De esta clase heredan *ChainedAttributeNode*, *ChainedArrayNode* y *ChainedMethodNode*. Cada una representa un tipo distinto de acceso y realiza las validaciones pertinentes. La cadena se valida recursivamente, por lo que si el nodo es el último de la cadena, retorna recursivamente su tipo hasta el primer nodo de la cadena.

ChainedAttributeNode valida que el atributo accedido exista y sea público, para invocar la validación del siguiente nodo si existe, pasándole el tipo del atributo o retorno del método.

ChainedArrayNode guarda también una expresión del índice accedido. Esta clase valida que el atributo accedido exista, sea público y sea de tipo *Array*, mientras que valida que la expresión del índice es de tipo I32. Si existe un siguiente nodo, invoca su validación pasándole el tipo de dato de los elementos del arreglo.

ChainedMethodNode guarda el nombre del atributo y una lista de expresiones representando los argumentos del método. En la validación comprueba que el método exista en la clase y que la cantidad y tipo de argumentos coincidan con la definición formal del método. Si existe un siguiente nodo, invoca su validación pasándole el tipo de dato de retorno del método.

3.2.12 Tipos de Variable

Se implementó la clase abstracta *Type* para representar a los distintos tipos de datos de TinyRust+ a la hora de declarar el tipo de una variable, argumento, atributo o tipo de retorno de un método. Esta clase únicamente posee un atributo que registra el identificador del tipo de dato de TinyRust+. Por ejemplo, “Str”, “Char”, “I32”, etc.

Los elementos de esta clase son utilizados tanto por la tabla de símbolos (para registrar el tipo de sus entidades) como por el AST (para resolver el tipo de expresiones).

Las clases *Array, PrimitiveType, Void* y *ClassType* heredan de *Type*. Las instancias de la clase *Array* incluyen también el tipo de dato contenido en el arreglo (por

ejemplo, “Array-Str” o “Array-I32”). El tipo *ClassType* representa a cualquier clase de TinyRust+ definida por el usuario, simplemente guarda el identificador de la clase, pero mantiene la interfaz consistente con *Type*, a diferencia de *ClassEntry*.

De la clase abstracta *PrimitiveType* extienden *Bool, Char, I32* y *Str,* los cuatro tipos primitivos de TinyRust+. Lo único que agrega esta clase intermedia es la posibilidad de especificar el valor por defecto y el valor real de las expresiones literales de este tipo.

3.2.13 Otras clases e interfaces

Para la implementación de la tabla de símbolos de TinyRust+ se crearon también algunas clases e interfaces que no se corresponden directamente con ninguna entidad de TinyRust+, pero que facilitan la extensibilidad y legibilidad del código.

La interfaz *TableElement* es implementada por todos las entidades de la tabla de símbolos y exige que su clase implemente el método *toJson*. El método *toJson* permite construir de manera recursiva la representación en formato JSON de la tabla de símbolos, ya que cada elemento llama al método *toJson* de sus atributos para armar su propia representación.

La interfaz *Node* es implementada por todos los nodos del AST y exige que su clase implemente los métodos *toJson* y *validate*, para garantizar la recursividad de la validación del árbol y generación del JSON.

La clase genérica *TableList* hereda de *HashMap* y sirve para almacenar conjuntos de entidades del mismo tipo de la tabla de símbolos, por ejemplo: lista de métodos, atributos o clases. A su vez, esta clase agrega funcionalidad a *HashMap* ya que implementa también el método *toJson*.

La clase genérica TreeList hereda de LinkedList y sirve para almacenar listas de nodos del mismo tipo del AST. A diferencia de TableLIst, TreeList conserva el orden de sus elementos, pero su complejidad de acceso no es constante. A su vez, esta clase agrega funcionalidad a *LinkedList* ya que implementa también los métodos *toJson* y *validate*, que consisten en llamar al método homónimo de cada uno de sus elementos.

La clase Location simplemente guarda un número de línea y columna del archivo de entrada donde se encontró un Token, usada para mostrar correctamente la información en los errores semánticos. Cada entidad de la TS y nodo del AST incluyen un atributo de este tipo.

3.3 Errores

Para el manejo de errores de cada etapa de análisis se creó una clase abstracta distinta que hereda de la clase Exception de Java. Los errores semánticos heredan de

SemanticalError. Una nueva instancia de esta clase de excepción espera recibir el número de línea y columna donde ocurrió el error junto con una descripción. A la hora de mostrar el mensaje de error, este incluirá estos tres datos.

En el caso del análisis semántico de declaraciones, para cada tipo de error distinto que pueda ocurrir durante el análisis se creó una clase distinta que herede de *SemanticalError*. Los distintos tipos de error posibles en esta etapa se explican en la sección 2.2. Para poder indicar la línea y columna donde ocurrió un error de consolidación, cada entidad de la tabla de símbolos guarda registro de la línea y columna donde fue declarada. Como caso particular, en la implementación se incluye un error de tipo *InternalError*. Esta es una excepción que indica un error de programación, se utiliza en situaciones donde es prolijo manejar una posible excepción, pero donde teóricamente no es posible que se cumplan las condiciones para lanzar el error. Por ejemplo, si se invocara la función para agregar un argumento al método actual, pero no se estuviera analizando ningún método actualmente.

Para el análisis semántico de sentencias se siguió un enfoque similar, para lo cual se creó la clase *ASTError* que hereda de *SemanticalError*. A su vez, se creó una clase de error distinta que herede de *ASTError* para los distintos tipos de error de validación del AST explicados en la sección 2.2. Para poder indicar la línea y columna donde ocurrió el error, cada entidad del AST guarda registro de la línea y columna donde fue declarada. Para esta etapa también se implementó la clase *InternalError* para detectar errores de programación tempranamente.

3.4 Casos de prueba

En el código del proyecto se incluyen los casos de prueba que se utilizaron para poner a prueba el funcionamiento de cada etapa de análisis. Dichos archivos se pueden encontrar en el directorio *tests* del proyecto, donde se encuentra una carpeta por cada etapa de análisis, conteniendo archivos de prueba con casos exitosos y casos de fallo.

La lista de tests del analizador semántico de sentencias consiste en más de 40 archivos de prueba distintos. Los primeros 3 archivos representan casos de prueba exitosos: mientras que los ejemplos de test número 0 y 2 fueron provistos por la cátedra, el ejemplo de test número 1 hace un uso exhaustivo de las posibilidades de uso válidas de TinyRust+.

El resto de archivos de prueba son casos de falla, en los que se espera que el compilador arroje cierto error en un nodo del AST en particular que no respete la semántica del lenguaje. Se crearon varios escenarios de prueba para cada tipo de error de esta etapa documentado en secciones previas. Cada archivo incluye al final un comentario especificando en qué lugar del archivo ocurrió el error y una explicación amigable con el usuario de la restricción semántica que se está incumpliendo.

3.5 Compilado y ejecución

Para compilar el proyecto, deben ejecutarse los siguientes comandos desde el directorio raíz del proyecto.

Paso 1: compilar.

```
javac lexic_analyzer/*.java syntactic_analyzer/*.java
semantic_analyzer/symbol_table/*.java semantic_analyzer/ast/*.java
semantic_analyzer/types/*.java util/*.java *.java
```

Paso 2: generar el jar.

```
jar cfm etapa4.jar Manifest.txt *
```

Para ejecutar el proyecto debe ejecutarse el siguiente comando:

```
java -jar etapa4.jar <ARCHIVO_FUENTE>
```

Es obligatorio especificar la ruta del archivo fuente que TinyRust+ analizará. En el directorio *tests* se pueden hallar archivos de prueba para alimentar el comando. Por ejemplo:

```
java -jar etapa4.jar ./tests/semantic_sentences/"test (0).rs" ./output.o
```

El comando generará hasta tres archivos de salida.

Especificar un archivo de salida es opcional. En caso de ser especificado, se escribirá el resultado del análisis en dicho archivo. Este resultado puede ser un mensaje de éxito o la información de un error, en caso de que el análisis no se haya podido completar correctamente. Si el archivo existe previamente se escribirá la salida al final del archivo, caso contrario será creado por el compilador. En caso de no especificarse este parámetro, la salida se mostrará por pantalla.

Solo en caso de que el análisis haya sido exitoso, se crearán dos archivos de igual nombre que el archivo de entrada pero con distinta extensión. El archivo de extensión *.ts.json* guarda una representación en formato JSON de la tabla de símbolos con todo su contenido, mientras que el archivo con extensión *.ast.json* guarda una representación en formato JSON del árbol sintáctico abstracto del archivo.

3.6 Ejemplos de salida

Se muestra a continuación un ejemplo de salida de las representaciones JSON de la TS y el AST para un archivo de entrada dado.

El archivo de entrada tomado de ejemplo es el siguiente:

```
class Derivada : Base {
   I32: x;
}

class Base {
   pub Str: s;
   I32: a;
   fn m1(I32: b) -> void {
   }
}

fn main() {
   Base: b1;
   b1 = new Derivada();
   (b1.m1(2 * 2));
}
```

La representación de la TS contiene una lista de clases y por cada clase contiene su nombre y listas de métodos y atributos. En la lista de clases se incluye una entrada para el método *main* y una entrada para cada clase predefinida. Cada método muestra su nombre y sus listas de argumentos y variables junto con sus nombres, posición y tipo. A continuación se pueden ver fragmentos de esta representación JSON.


```
{
   "nombre": "./tests/semantic_sentences/test-ejemplo.rs",
   "clases": [
      {
            "nombre": "Str",
            "heredaDe": "Object",
            "heredable": false,
            "atributos": [],
            "constructor": {
                  "argumentosFormales": []
            },
            "metodos": [
                  {
                        "nombre": "length",
                        "posicion": 1,
                        "static": false,
                        "tipoRetorno": "I32",
                        "argumentosFormales": [],
                        "variablesBloque": []
                  },
                  {
                        "nombre": "concat",
                        "posicion": 2,
                        "static": false,
                        "tipoRetorno": "Str",
                        "argumentosFormales": [
                              {
```

```
                                        "nombre": "s",
                                        "posicion": 1,
                                        "tipo": "Str"
                                }
                        ],
                        "variablesBloque": []
                },
                {
                        "nombre": "substr",
                        "posicion": 3,
                        "static": false,
                        "tipoRetorno": "Str",
                        "argumentosFormales": [
                                {
                                        "nombre": "i",
                                        "posicion": 1,
                                        "tipo": "I32"
                                },
                                {
                                        "nombre": "l",
                                        "posicion": 2,
                                        "tipo": "I32"
                                }
                        ],
                        "variablesBloque": []
                }
        ]
},
{
        "nombre": "Array",
        "heredaDe": "Object",
        "heredable": false,
        "atributos": [],
        "constructor": {
                "argumentosFormales": []
        },
        "metodos": [
                {
                        "nombre": "length",
                        "posicion": 1,
                        "static": false,
                        "tipoRetorno": "I32",
                        "argumentosFormales": [],
                        "variablesBloque": []
                }
        ]
},
{
        "nombre": "Bool",
        "heredaDe": "Object",
        "heredable": false,
```

```
            "atributos": [],
            "constructor": {
                    "argumentosFormales": []
            },
            "metodos": []
    },
    /* [...más clases predefinidas: IO, Object, I32, Char, Bool ...] */
    {
            "nombre": "main",
            "heredaDe": null,
            "heredable": false,
            "atributos": [],
            "constructor": null,
            "metodos": [
                    {
                            "nombre": "main",
                            "posicion": 1,
                            "static": true,
                            "tipoRetorno": "void",
                            "argumentosFormales": [],
                            "variablesBloque": [
                                    {
                                            "nombre": "b1",
                                            "posicion": 1,
                                            "tipo": "Base"
                                    }
                            ]
                    }
            ]
    },
    {
            "nombre": "Derivada",
            "heredaDe": "Base",
            "heredable": true,
            "atributos": [
                    {
                            "nombre": "a",
                            "posicion": 2,
                            "public": false,
                            "tipo": "I32"
                    },
                    {
                            "nombre": "s",
                            "posicion": 3,
                            "public": true,
                            "tipo": "Str"
                    },
                    {
                            "nombre": "x",
                            "posicion": 1,
                            "public": false,
```

```
                        "tipo": "I32"
                }
        ],
        "constructor": {
                "argumentosFormales": []
        },
        "metodos": [
                {
                        "nombre": "m1",
                        "posicion": 1,
                        "static": false,
                        "tipoRetorno": "void",
                        "argumentosFormales": [
                                {
                                        "nombre": "b",
                                        "posicion": 1,
                                        "tipo": "I32"
                                }
                        ],
                        "variablesBloque": []
                }
        ]
},
{
        "nombre": "Base",
        "heredaDe": "Object",
        "heredable": true,
        "atributos": [
                {
                        "nombre": "a",
                        "posicion": 2,
                        "public": false,
                        "tipo": "I32"
                },
                {
                        "nombre": "s",
                        "posicion": 1,
                        "public": true,
                        "tipo": "Str"
                }
        ],
        "constructor": {
                "argumentosFormales": []
        },
        "metodos": [
                {
                        "nombre": "m1",
                        "posicion": 1,
                        "static": false,
                        "tipoRetorno": "void",
                        "argumentosFormales": [
```

```
                                        {
                                            "nombre": "b",
                                            "posicion": 1,
                                            "tipo": "I32"
                                        }
                                    ],
                                    "variablesBloque": []
                                }
                            ]
                        }
                    ]
                }
```


La representación JSON del AST suele ser más extensa, dependiendo de la complejidad de las sentencias del programa. El JSON comienza con el nombre del archivo y su lista de clases. La lista de clases incluye una entrada para el método main y una para cada clase predefinida. Para cada clase únicamente se muestra su nombre y lista de métodos. Luego cada método muestra sólo su nombre y su bloque. Para el bloque se muestra la lista de sentencias que lo componen: cada sentencia incluye el tipo de sentencia y datos relevantes de su composición como expresiones, accesos y sub-bloques. Para las expresiones se muestra su tipo esperado, tipo resuelto, accesos encadenados, nombres de variables y métodos accedidos, valores de literales y, en la mayoría de los casos, más expresiones anidadas (cuyo rol siempre se especifica como *key* del atributo JSON).


```
{
  "nombre": "./tests/semantic_sentences/test-ejemplo.rs",
  "clases": [
    {
      "nombre": "Array",
      "metodos": [
        {
          "nombre": "length",
          "Bloque": {
            "tipo-sentencia": "Bloque",
            "sentencias": []
          }
        }
      ]
    },
    {
      "nombre": "Object",
      "metodos": []
    },
     /* [...más clases predefinidas: IO, Str, I32, Char, Bool ...] */
    {
      "nombre": "Derivada",
      "metodos": []
    },
    {
```

```
      "nombre": "Base",
      "metodos": [
         {
            "nombre": "m1",
            "Bloque": {
               "tipo-sentencia": "Bloque",
               "sentencias": []
            }
         }
      ]
   },
   {
      "nombre": "main",
      "metodos": [
         {
            "nombre": "main",
            "Bloque": {
               "tipo-sentencia": "Bloque",
               "sentencias": [
                  {
                     "tipo-sentencia": "Asignacion",
                     "izquierda": {
                        "tipo": "variable",
                        "nombre-variable": "b1",
                        "tipo-resuelto": "Base",
                        "encadenado": null
                     },
                     "derecha": {
                        "tipo": "llamada constructor",
                        "clase": "Derivada",
                        "argumentos": [],
                        "tipo-resuelto": "Derivada",
                        "encadenado": null
                     }
                  },
                  {
                     "tipo-sentencia": "Sentencia-Simple",
                     "expresion": {
                        "tipo": "variable",
                        "nombre-variable": "b1",
                        "tipo-resuelto": "void",
                        "encadenado": {
                           "tipo": "acceso metodo encadenado",
                           "nombre-metodo": "m1",
                           "argumentos": [
                              {
                                 "tipo": "expresion binaria",
                                 "operador": "*",
                                 "tipo-de-retorno": "I32",
                                 "tipo-de-operandos": "I32",
                                 "operando-derecha": {
```

```
                                    "tipo": "literal",
                                    "valor": "2",
                                    "tipo-dato": "I32"
                                },
                                "operando-izquierda": {
                                    "tipo": "literal",
                                    "valor": "2",
                                    "tipo-dato": "I32"
                                }
                            }
                        ],
                        "encadenado": null
                    }
                }
            }
        ]
    }
  }
 ]
}.
```


4. Conclusiones

Habiendo completado la implementación del analizador semántico, la construcción de la tabla de símbolos y el árbol sintáctico abstracto queda completado el Frontend de este compilador de TinyRust+.

Luego de pasar por esta última etapa de análisis, me permito afirmar que el lenguaje es más completo de lo que puede aparentar a simple vista por su sintaxis limitada. Las validaciones que se realizaron en esta etapa representan en realidad la esencia de este pequeño lenguaje, ya que son lo que diferencia a TinyRust+ de otros lenguajes del mismo paradigma y de otros paradigmas.

A partir de ahora, tan solo queda generar el código de bajo nivel que traiga a la vida los programas escritos en TinyRust+. Ya es el último paso, pero esto no implica que por ser el último sea un desafío fácil.

5. Referencias

1. JSON. ECMA-404 The JSON Data Interchange Standard. https://www.json.org/json-es.html